

Apellido y Nombre: [REDACTED]

Hojas: 3

1	2	3					TOTAL
1/3	1/3	1/3					3/3

- 1) Escriba en la siguiente grilla (en decimal) las micro-instrucciones necesarias para ejecutar:

$$[(AX \text{ and } AMask) - PC] = [DX + SP]$$

Considere que la estructura de la micro-instrucción horizontal es:

AMUX	COND	ALU	SH	MBR	MAR	RD	WR	ENC	C	B	A	ADDR
		0					1	1	13	2		ac := DX + SP
		2		1	1				1			mar := ac, rd
						1						rd
		3					1	4			0	tr := inv(PC) } -PC
		0					1	4	4	6		tr := tr + 1
		1					1	1	10	8		ac := AX & AMASK
		0					1	4	4	1		tr := tr + ac
				1		1			4			mar := tr, wr
						1						wr

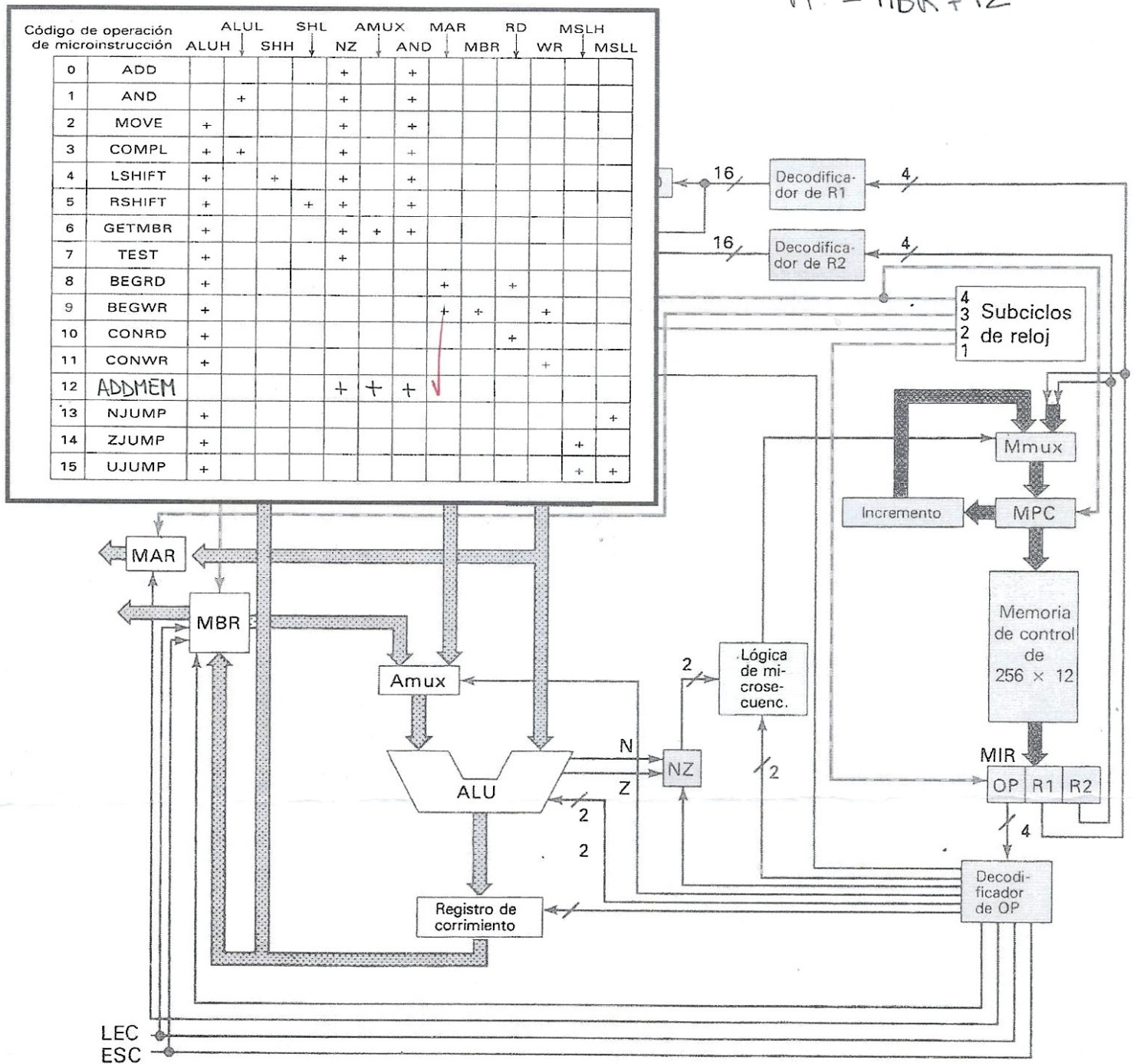
AMUX	COND	ALU	SH	MBR, MAR, RD, WR, ENC	REGISTROS
0 = Buffer A 1 = MBR	0 = No salta 1 = Si N==1 2 = Si Z==1 3 = Salta siempre	0 = A + B 1 = A & B 2 = A 3 = ~A	0 = No desplaza 1 = alu >> 1 2 = alu << 1 3 = (no usado)	0 = No 1 = Si	PC=0, AC=1, SP=2,... TIR=4,... K ₀ =5, K ₁ =6, K ₂ =7 AMASK=8,... AX=10, BX=11,... CX=12 DX=13

- 2) Dada la arquitectura vertical propuesta por Tanenbaum, se quiere implementar la nueva microinstrucción **ADDMEM** (cuya funcionalidad es $r1 := MBR + r2$) correspondiente al código 12 del Decodificador de OP.

- Escriba cómo quedaría modificada la matriz representativa del Decodificador de OP.
- ¿El agregado de esta nueva microinstrucción es útil, inútil o intrascendente para la programación del micro-programa? Justifique su respuesta.
- Los saltos dentro del micro-programa vertical ¿son absolutos o relativos? Justifique su respuesta.

NOTA: Se exige un mínimo del 40% de desarrollo correcto de cada tema para comenzar a tener puntaje en dicho tema.
Responder cada tema en forma completa, precisa y con letra legible.

$$r1 = MBR + r2$$



3) En una arquitectura de **JAVA-VM**:

- Describe para qué se usan registros: LV, CPP, TOS, OPC y H.
- Caracterice el mecanismo para acceder a la memoria RAM, afin de:
 - modificar un dato
 - recuperar una instrucción

En **ambos casos** identifique los registros que intervienen

- Caracterice la ejecución del micro-programa
 - cuando el campo J de la micro-instrucción es "000"
 - cuando el campo JAMZ vale "1"

MIR:

Addr	J	J	J	ALU	C	F	R	W	
	a	a	a			e	e	r	
	m	m	m			t	a	i	B
	N	Z	P			c	d	t	
			C			h	e	e	

②

b) Considero que el agregado de esta instrucción es inútil porque se podría llegar a lo mismo utilizando otras instrucciones. Podría obtener el valor de MBR (GETMBR) guardarlo en un registro (R1 por ejemplo) y luego sumarle a R1 R2 utilizando la instrucción ADD. De este modo ya no necesitaría la instrucción que acabo de representar en la matriz representativa del Decodificador de OP. Sin embargo, aunque se puede obtener el mismo resultado combinando otras microinstrucciones, puede que hacerlo en una sola sea más eficiente. De todos modos no evita el hecho de que estoy agregando un nuevo código de operación en el Decodificador de OP.

c) Los saltos dentro del microprograma vertical son absolutos, ya que cuando se tiene un código de operación que representa un salto (NJUMP, ZJUMP, VJUMP) los operandos R1 y R2 se combinan para formar un campo de 8 bits para indicar la dirección dentro del microprograma a la que se desea saltar. De este modo, no es que se indica un valor relativo a la posición de la microinstrucción ^{actual} para moverse (+n, -n) sino que se indica directamente la dirección a la que hay que saltar.

Según el código de operación se evalúan diferentes cuestiones dentro de la "Lógica de microsecuencia".

miento" (si N es 1, si Z es 1, o puede ser también un salto incondicional). Si se cumple la condición de salto se toma el valor del Mmux en la ruta derecha (la dirección indicada por los 8 bits) y sino se incrementa en 1 el valor del MPC y se toma la siguiente microinstrucción (secuencialmente) (para izquierda del multiplexor: $MPC+1$)



(3) a)

• LV (Local Variable): este registro apunta a la base del marco donde se encuentran las variables locales. Las variables locales suelen "apilarse" o aparecer en este marco cuando se invoca el procedimiento (que las contiene y utiliza) y "desaparecer" cuando el mismo finaliza y se vuelve a donde se lo invocó. ✓

• CPP: (Constant Pool Pointer): CPP es un registro que apunta a la base de la pila de constantes. Las constantes no cambian su valor durante la ejecución del programa.

• TOS (Top of Stack): mientras que el registro SP (stack pointer) apunta al tope de la pila, TOS contiene el valor ~~que~~ hay en la dirección apuntada por SP (el dato).

• OPC (Old Program Counter): este registro apunta a la dirección anterior a la que apunta el PC actualmente. Es útil para almacenar la dirección de retorno (cuando se invoca a una subrutina). También se podría llegar a utilizar para hacer operaciones. ✓

• H (Held): este registro alimenta la pila izquierda de la ALU. En caso de que se desee alimentar a la pila izquierda de la ALU con un valor que se encuentra en los registros, el mismo debería hacerse pasar por la

para derecha de la ALU y guardarse en H, que constantemente alimenta la pata izquierda de la ALU

b) i) Si se desea modificar un dato va a ser necesario utilizar el MAR (Memory Address Register) y el MDR (Memory Data Register). Básicamente debe colocarse en el MAR la dirección de memoria sobre la que se quiere escribir (para modificar un dato en la RAM) y en el MDR se debería colocar el dato que se desea guardar (por el cual se va a reemplazar lo que esté en ese momento en esa dirección en la RAM). Se inicia la operación de escritura, y así queda modificado el dato en la RAM.

Si por alguna razón antes o después quisiera leer el dato de la RAM debería hacer una operación de lectura indicando la dirección que quiero leer en MAR y obtendré el dato en el MDR.

ii) Para recuperar una instrucción uso los registros PC y MBR. (Program Counter y Memory Buffer Register). ~~Debo~~ En este caso se cargan en los 8 bits más bajos del MBR la instrucción que está siendo apuntada por PC. Por lo tanto PC debería tener cargada una dirección de memoria que contenga instrucciones (el programa).

c) i) En la arquitectura JAVA-VM no existe un procesamiento que incremente automáticamente el MPC (como si lo tiene por ejemplo la arquitectura vertical). De este modo todas las instrucciones JAVA-VM tienen

un campo llamado NEXT-ADDRESS que indica la siguiente microinstrucción a ejecutar. En el caso de que J sea "000" este campo se copia tal cual al MPC. Es decir, la siguiente instrucción es puramente la indicada por el campo NEXT-ADDRESS.

ii) En el caso de que JAMZ valga 1, se va a utilizar el componente "Bit Alto" y se hará un OR entre el flipflop Z y NEXT-ADDRESS. De este modo si Z quedó en 1 por el resultado que pasó por la ALU anteriormente, al hacer un OR (utilizando el componente 0) terminaría siendo la siguiente microinstrucción a ejecutar NEXT-ADDRESS OR 0x100. De este modo se realiza un salto cuando el valor de Z es 0.